UTEC
Universidad Tecnológica

Patrón Abstract Factory

El **Patrón Abstract Factory** es un patrón de diseño creacional que se utiliza para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Este patrón proporciona una interfaz para crear un conjunto de objetos que pertenecen a una misma familia y que están destinados a funcionar juntos. Es especialmente útil cuando se requiere garantizar que los productos de una familia sean compatibles entre sí.

Creación de familias de objetos relacionados

El propósito principal del patrón Abstract Factory es permitir la creación de familias de objetos relacionados. Esto significa que se puede crear un conjunto de objetos que comparten una relación lógica o funcional y que deben ser usados juntos. Por ejemplo, en una aplicación que tiene diferentes interfaces gráficas de usuario (GUI), podrías tener familias de botones, menús, y ventanas que cambian dependiendo del sistema operativo (Windows, macOS, Linux). El patrón Abstract Factory asegura que los botones, menús, y ventanas que pertenecen a la misma familia (por ejemplo, familia "Windows") funcionen bien juntos.

Estructura del Patrón Abstract Factory

La estructura básica del patrón Abstract Factory involucra los siguientes componentes:

- **Abstract Factory**: Es la interfaz que declara los métodos para crear los productos abstractos (por ejemplo, `crearBoton()`, `crearMenu()`).
- **Concrete Factory**: Implementa la interfaz Abstract Factory y es responsable de crear los productos concretos pertenecientes a una familia específica (por ejemplo, `BotonWindows`, `MenuWindows`).
- **Producto Abstracto**: Es una interfaz o clase abstracta que define un tipo de producto (por ejemplo, `Boton`, `Menu`).
- **Producto Concreto**: Son las implementaciones concretas de los productos abstractos (por ejemplo, `BotonWindows`, `MenuWindows`).

Ejemplo: Vehículos y Componentes

Imaginemos que estamos desarrollando una aplicación que fabrica diferentes tipos de vehículos (coches y motos) y que cada vehículo tiene componentes específicos como

ruedas y motores. Queremos asegurar que los componentes de cada tipo de vehículo sean compatibles entre sí.

1. Interfaces de los Productos Abstractos

```
public interface Rueda {
    void fabricar();
}

public interface Motor {
    void ensamblar();
}
```

Estas interfaces representan los componentes abstractos que queremos crear (ruedas y motores).

2. Implementaciones Concretas de los Productos

Ahora implementamos estas interfaces para diferentes tipos de vehículos.

```
public class RuedaCoche implements Rueda {
    @Override
    public void fabricar() {
        System.out.println("Fabricando rueda de coche.");
    }
}

public class RuedaMoto implements Rueda {
    @Override
    public void fabricar() {
        System.out.println("Fabricando rueda de moto.");
    }
}

public class MotorCoche implements Motor {
    @Override
    public void ensamblar() {
        System.out.println("Ensamblando motor de coche.");
    }
}
```

```java
public class MotorMoto implements Motor {
    @Override
    public void ensamblar() {
        System.out.println("Ensamblando motor de moto.");
    }
}
```

Estas clases representan las implementaciones concretas de las ruedas y motores para coches y motos.

3. Interfaz Abstracta para la Fábrica

Definimos una interfaz para las fábricas que crearán estos componentes.

```java
public interface VehiculoFactory {
    Rueda crearRueda();
    Motor crearMotor();
}
```

4. Implementaciones Concretas de la Fábrica

Ahora creamos las fábricas concretas para coches y motos.

```java
public class CocheFactory implements VehiculoFactory {
    @Override
    public Rueda crearRueda() {
        return new RuedaCoche();
    }

    @Override
    public Motor crearMotor() {
        return new MotorCoche();
    }
}

public class MotoFactory implements VehiculoFactory {
    @Override
```

UTEC
Universidad Tecnológica

```
    public Rueda crearRueda() {
        return new RuedaMoto();
    }

    @Override
    public Motor crearMotor() {
        return new MotorMoto();
    }
}
```

Estas fábricas se encargan de crear los componentes específicos para coches y motos.

5. Uso del Patrón Abstract Factory

Finalmente, utilizamos las fábricas para crear los componentes de un vehículo.

```
public class Ensamblaje {
    private Rueda rueda;
    private Motor motor;

    public Ensamblaje(VehiculoFactory factory) {
        rueda = factory.crearRueda();
        motor = factory.crearMotor();
    }

    public void ensamblarVehiculo() {
        rueda.fabricar();
        motor.ensamblar();
    }

    public static void main(String[] args) {
        VehiculoFactory factory;

        // Cambiar aquí para crear los componentes de un coche o una
moto
        factory = new CocheFactory();
        // factory = new MotoFactory();

        Ensamblaje ensamblaje = new Ensamblaje(factory);
        ensamblaje.ensamblarVehiculo();
```

```
    }
}
```

Explicación

En este ejemplo, el patrón **Abstract Factory** asegura que, si eliges fabricar un coche, todas las piezas creadas (ruedas y motor) son compatibles con un coche. De la misma forma, si eliges fabricar una moto, todos los componentes serán compatibles con una moto. Esto garantiza que no mezcles accidentalmente componentes de diferentes tipos de vehículos, manteniendo la coherencia en tu aplicación.

Este enfoque es particularmente útil cuando se desea poder cambiar la familia de productos (en este caso, el tipo de vehículo) sin cambiar el código que usa esos productos.

Comparación con Factory Method

Aunque tanto el **Factory Method** como el **Abstract Factory** son patrones creacionales y se utilizan para crear objetos, existen diferencias clave entre ellos:

- **Grado de Complejidad**: El Factory Method es menos complejo y más específico. Está diseñado para crear un solo tipo de producto. En cambio, Abstract Factory está pensado para crear familias de productos relacionados.
- **Estructura**: El Factory Method se centra en la creación de un solo objeto mediante un método, mientras que el Abstract Factory proporciona una interfaz para crear una familia de productos relacionados mediante múltiples métodos.
- **Relación entre Productos**: En Factory Method, no hay una relación explícita entre los productos creados por diferentes métodos fábrica. En Abstract Factory, los productos creados por una fábrica concreta están diseñados para trabajar juntos.
- **Uso**: Usa Factory Method cuando quieras crear un solo producto o una variación específica de un producto. Usa Abstract Factory cuando necesites crear un grupo o familia de productos relacionados, y necesites asegurarte de que estos productos son compatibles entre sí.

Resumen

El **Patrón Abstract Factory** es útil para crear familias de objetos relacionados que deben trabajar juntos. Es más complejo que el Factory Method pero también más potente cuando se necesita manejar la creación de varios objetos relacionados de manera cohesiva. Este patrón asegura que una familia de productos sea creada de forma consistente y

manteniendo la compatibilidad entre sus componentes, lo que es fundamental en aplicaciones con diferentes configuraciones o versiones de productos.